

Asymptotic Analysis after Six Review Waves

Operand-preserving validation, an all-depth inverse counterexample,
and exact-rational Lerch certificates

Technical review prepared for Vladimir Reshetnikov

10 September 2026

Source snapshot:

`efa1aeec4845a9c35e140963a0333d0c9ec33b05`

Abstract

This incremental audit identifies a correctness defect in the exact-core inverse constructor: it checks target independence after adding the core and perturbation, although it subsequently uses the operands separately. Cancellation can therefore admit target-dependent model data into a theorem whose constants must be fixed. For a particularly simple input, the total forward function is exactly $1/x$, but the source-derived marker approximations alternate between 0 and $2/y$ while the claimed remainder improves by one power at every depth. A second family has a genuinely vanishing perturbation ratio and still violates the reported scale. The report gives an all-depth proof, a narrowly scoped repair, and executable independent checks.

A separate development contribution constructs a hierarchy of rational lower and upper bounds for $\Phi(q, 1, a)$ using the two Meixner measures naturally associated with the defining sum. The implementation evaluates two three-term recurrences: it needs neither a moment matrix nor quadrature nodes, a special-function call, or floating-point arithmetic. Positivity, error identities, nesting and explicit error caps are proved. Python code is independently tested; an equivalent Wolfram Language prototype is supplied.

The comparison baseline consists of the six review indexes, the maintained register, and focused original passages relevant to the proposed deltas. Existing findings are not presented as new. Evidence is deliberately separated: source inspection, mathematical proof, isolated Wolfram 15 evaluations, and independent Python tests do not constitute an upstream package or Mathics acceptance run.

Deliverable boundary. One new source-established correctness finding; one concrete algorithmic development proposal. No new benchmark result, complete compatibility claim, integrated patch, or full-suite acceptance is asserted.

Contents

1	Executive assessment and evidence boundary	3
1.1	What should change first	3
1.2	What actually ran	3
1.3	Review coverage	4
2	Novelty relative to the existing review corpus	4
2.1	N01 is a different admission defect	4
2.2	E01 is an algorithm, not a repeated roadmap request	5
3	N01: cancellation defeats fixed-operand validation	5
3.1	The source path	5
3.2	A public witness with a trivial true inverse	6
3.3	The marker equation exposes the error	6
3.4	The failure survives a genuinely small perturbation ratio	7
3.5	The valid fixed-data control	8
3.6	Native isolation: what is confirmed	8
4	Repair, regression design, and dual-runtime implications	8
4.1	Minimal semantic change	8
4.2	Why an output-only repair is inadequate	9
4.3	Regression expectations	9
4.4	Mathics: common source is not common observed behavior	9
5	E01: a rational Lerch certificate hierarchy	10
5.1	The bounded target	10
5.2	Two positive measures, not a moment matrix	10
5.3	A general resolvent identity	10
5.4	The lower and upper endpoints	11
5.5	Closed recurrence coefficients	11
5.6	Denominator positivity without approximate roots	12
5.7	Nested bounds	12
5.8	Explicit error caps and asymptotic order	13
5.9	Concrete exact example	13
6	Implementation contracts and tests for E01	14
6.1	A small API with explicit semantics	14
6.2	Why value bounds must not become series metadata accidentally	14
6.3	Independent validation paths	14
6.4	A test-harness correction, preserved in the evidence	15
6.5	Performance claims that are and are not supported	15
7	Other inspected candidates not counted as findings	15
8	Integration priorities and reproducibility	16
8.1	Recommended sequence	16
8.2	Archive layout	16
8.3	Commands	17
9	Conclusion	17
A	Source and evidence map	17

1 Executive assessment and evidence boundary

1.1 What should change first

N01: validate the two operands before combining them. The highest-priority change is in `corePerturbationConstruct`, in `src/Kernel/CorePerturbation.wl`. The constructor tests `FreeQ[core + perturbation, y]`, not `FreeQ[{core, perturbation}, y]`. Its parser then keeps separate core and perturbation rows, extracts an offset from the core, constructs an exact core inverse, and computes a marker expansion with a fixed-data majorant. Those operations depend on facts about the operands that the sum cannot preserve. The public error message already requires no target symbol in the forward data; the implementation enforces a weaker condition.[1]

This is not merely an overly optimistic numerical estimate. For the witness in section 3, the error divided by the stored remainder scale tends to infinity. No finite hidden big- O constant can repair the result. Its source and target domains are nonempty and contain the entire required eventual ray. The bad result is also not dependent on a difficult special-function identity.

E01: implement a specific rational certificate family for the Lerch slice $s = 1$. The repository already has proposals for Lerch remainder bounds and exact moments. This report does not repeat either request. Its additional contribution is the explicit two-measure Gauss/Radau construction in section 5: a nested rational interval, a positive denominator formula, closed error caps, and a linear number of exact recurrence steps. It supplies a bounded implementation that can be tested independently of either symbolic kernel.[9, 10]

1.2 What actually ran

Evidence class	Actual scope
Repository reads	Commit-pinned GitHub connector reads of the modules and review material described below. No local checkout was obtained.
Mathematical analysis	Exact all-depth counterexample and fixed-data controls; proofs of the rational certificate construction and its error identities.
Wolfram 15 isolation	A transcribed evaluation of the upstream marker-term recurrence, the relevant cancellation and realness predicates, and five coefficients; a separate evaluation of the proposed rational recurrence for four orders, with four native Lerch comparisons.
Python execution	24 unittest methods, zero failures and zero errors in the final run. The methods include exact-rational models, independent defining-sum and moment oracles, and synthetic patch-emitter fixtures.
Not executed	The complete original package, the patched package, the supplied MUnit regressions, the complete supplied WL wrapper file, and all Mathics runs.

The available remote kernel reported 15.0.0 for Linux x86 (64-bit) (May 6, 2026). Package-loading requests failed at the service boundary. Container downloads also failed, and no local Wolfram or Mathics executable was available. These are environment limitations, not package test failures. Isolated native calculations are consequently marked as such throughout this article. In particular, the full public N01 call is a *source-predicted manifestation*, not a fabricated native transcript.

The final Python count must not be added to any historical acceptance count in the repository. Likewise, the 144 containment subcases in one test method are not 144 extra unittest methods. The machine-readable scope record and test log are included in `evidence/`.

1.3 Review coverage

The audit concentrated on semantic admission, coordinate transformations, remainder construction, observable operations, exact arithmetic, and portable proof boundaries. It read the complete exact-core module and selected portions of the general core, the source and inverse coordinate modules, ordinary and reciprocal-logarithmic operations, Fourier coefficients, Dirichlet special functions, native special-function admission, inverse syntax and branches, certificates, and the Mathics assumption prover. The six wave indexes and the maintained status register were used to screen candidate findings. Relevant original sections of reports 49 and 50 were read for the Lerch proposal.

This is a bounded source review, not a statement that every source line, every retained article, or every test has been re-executed. Unread portions and unsupported runtimes are not silently converted into coverage. The accompanying `evidence/scope.json` and novelty ledger make that distinction explicit.

2 Novelty relative to the existing review corpus

The pinned review index retains 45 packages across six waves. Its own accounting separates report entries from distinct obligations and records overlaps rather than treating every report-local identifier as a separate bug. That is the appropriate baseline for an incremental review.[5, 6]

2.1 N01 is a different admission defect

The closest existing work is the target-dependent `SourceShift` finding in reports 46 and 50, the fixed-parameter scope principle recorded under C15, and report 43's source-symbol leakage through an exact core inverse. N01 is not another example of one of those call paths.[7, 8, 6]

Comparison	Additional obligation established here
Reports 46/50	N01 affects <code>AsymptoticCoreInverse</code> , not the exponential constructor's <code>SourceShift</code> option. The distinguishing mechanism is an aggregate dependency test that loses information through cancellation.
C15	Parameter capture during composition and admission of two independently varying operands are different operations. The repair here belongs before the core model is built, not in substitution or composition.
Report 43	The eliminated source symbol is not retained in N01's inverse. The target has entered the model through the separately stored core and perturbation. An output-only source-freedom check would pass this witness.

The exponential-core sibling is also an instructive local control: its variable admission uses a list containing the two operands. This is a precedent for the minimal repair, not a new request to redesign every chart abstraction. [2]

2.2 E01 is an algorithm, not a repeated roadmap request

Report 49 already develops signed mean–variance information for the Lerch remainder. Report 50 supplies an exact Eulerian moment table and discusses removing special-function dependence from those moments. The present proposal uses the same defining positive measure but computes a different object: two rational resolvent approximants of arbitrary finite order, evaluated by closed three-term recurrences.[9, 10]

The added obligations are the two shifted Meixner parameterizations, the positive-denominator formula, the exact error identities and caps, the nested interval proof, and a recurrence implementation requiring $O(n)$ arithmetic steps for a single order- n interval. Order one includes a familiar mean-based lower bound and is used as a control, not advertised as a new result. The useful additional family is the higher-order hierarchy and its constructive certificate.

The underlying orthogonal-polynomial and quadrature ideas are classical. The contribution is a concrete, proved and tested application to this repository’s bounded Lerch certificate lane, not a claim of mathematical priority.

3 N01: cancellation defeats fixed-operand validation

3.1 The source path

At the constructor boundary the current source contains:

```
validateInput[core + perturbation, limit];
If[x === y || ! FreeQ[core + perturbation, y],
  fail["InvalidVariables",
    "Use distinct source and target symbols, with no target symbol in the forward
    data."]];
```

The public overload separately restricts the coordinate slots to symbols. The problem is therefore not a missing symbol pattern in this particular argument form. It is the expression on which dependency is tested.[1]

The subsequent data flow is structurally different:

```
f0 = Simplify[core /. x -> coord["Substitution"], ...];
rr = Simplify[perturbation /. x -> coord["Substitution"], ...];
rows = parseFinite[f0, ...];
remainderRows = parseFinite[rr, ...];
offset = Total[Cases[rows, {0,c_} :> Coefficient[c,ell,0]]];
...
v = y - model["Offset"];
u0 = (v/a)^(1/p);          (* monomial exact core *)
```

Here the parser and inverse builder genuinely need the two operands, not merely their sum. The model records their separate finite supports, the perturbation’s relative source-power gap, and the core offset.[1]

For observable power one at a finite positive-side endpoint, the omitted-tail pair is constructed as

```
{canon[1 + (depth + 1) delta], (depth + 1) degree}
```

when the perturbation is nonzero. The stored majorant explicitly quantifies over *fixed data*: it asserts existence of finite constants and a threshold for an expansion in a small source-coordinate scale.[1]

3.2 A public witness with a trivial true inverse

Take the target y positive and eventually large, and use

```
AsymptoticCoreInverse[
  1/x + Abs[y]/2,
  -Abs[y]/2,
  {x, 0}, {y, n}
]
```

with a fixed nonnegative integer depth n . This call is deliberately written with `Abs[y]` rather than a bare y in the coefficient. Native Wolfram arithmetic can prove `Abs[y]` real without a separate assumption about y . The constructor prohibits target-dependent assumptions, so using an unproved-real bare coefficient would obscure the actual validation defect.

The total forward expression is

$$\left(\frac{1}{x} + \frac{|y|}{2}\right) - \frac{|y|}{2} = \frac{1}{x}.$$

Consequently the existing `FreeQ` test sees no target. On the positive source chart, both operands are finite power-log expressions. Their source powers are -1 for the leading core term and 0 for the perturbation. The computed relative power gap is therefore $\delta = 1$, and the logarithmic degree is zero. The leading amplitude is one. Nothing in those source-power checks detects that the coefficient at source power zero grows with the target.

On $y > 0$, put $A = y/2$. The exact core and perturbation are

$$f_0(u) = u^{-1} + A, \quad r(u) = -A, \quad u_0 = \frac{1}{y - A} = \frac{2}{y}.$$

The stored target-domain ingredients are satisfied for all sufficiently large y : $y - A > 0$, $u_0 \in \mathbb{R}$, and $0 < u_0 < 1/e$ whenever $y > 2e$. Thus the counterexample does not rely on evaluating outside the selected domain.

3.3 The marker equation exposes the error

Introduce the marker λ exactly as the algorithm does:

$$f_0(u) + \lambda r(u) = y.$$

For an arbitrary value $A = A(y)$ independent of the source coordinate,

$$u(\lambda) = \frac{1}{y - A + \lambda A} = \frac{u_0}{1 + \lambda A u_0}. \quad (1)$$

The degree- j marker coefficient is therefore

$$t_j = (-A)^j u_0^{j+1}, \quad j \geq 0. \quad (2)$$

This agrees with the source recurrence based on r^j/f'_0 and repeated application of $(1/f'_0) d/du$. It also follows directly by expanding the rational function in eq. (1); no numerical inference is needed.

For $A = y/2$ one has $Au_0 = 1$. The finite approximation is

$$S_n = \sum_{j=0}^n t_j = \begin{cases} 2/y, & n \text{ even}, \\ 0, & n \text{ odd}. \end{cases} \quad (3)$$

The true inverse is $u(1) = 1/y$, so the absolute error is $1/y$ at every depth. But the source constructs the remainder pair $(n+2, 0)$ and scale $u_0^{n+2} = (2/y)^{n+2}$. Their ratio is

$$\frac{|S_n - 1/y|}{u_0^{n+2}} = \frac{y^{n+1}}{2^{n+2}} \rightarrow \infty. \quad (4)$$

This refutes the asymptotic remainder at every fixed marker depth.

Theorem 3.1 (All-depth cancellation witness). *For each integer $n \geq 0$, the source path described above associates a remainder scale of order y^{-n-2} with an approximation whose error is exactly y^{-1} on an eventual positive target ray. Its claimed big-O statement is false.*

Proof. The source-power gap and observable power give $(n+2, 0)$. Equations (3) and (4) give the finite approximation, exact error, and unbounded normalized error. The domain check above establishes that the ray is admitted by the stored domain formula. $\square$

3.4 The failure survives a genuinely small perturbation ratio

The previous example has marker ratio $Au_0 = 1$. A repair that merely checks whether a marker ratio tends to zero would still miss the more fundamental fixed-data failure. Set instead

$$A(y) = \sqrt{y} \quad (y > 0),$$

represented in the call by `Sqrt[Abs[y]]`. Now

$$Au_0 = \frac{\sqrt{y}}{y - \sqrt{y}} \rightarrow 0.$$

The marker expansion is geometrically convergent at $\lambda = 1$ for all large enough y , and its truncations do improve. Nevertheless they do not improve at the rate stated in the stored source-power remainder.

Proposition 3.2 (General moving-offset identity). *Let $A(y) \geq 0$, $y > A(y)$, and $u_0 = (y - A(y))^{-1}$. For the degree- n finite sum from eq. (2),*

$$S_n - \frac{1}{y} = -\frac{(-Au_0)^{n+1}}{y}, \quad (5)$$

$$\frac{|S_n - 1/y|}{u_0^{n+2}} = A(y)^{n+1} \left(1 - \frac{A(y)}{y}\right). \quad (6)$$

If $A(y) = y^\sigma$ with $0 < \sigma < 1$, the normalized error diverges like $y^{\sigma(n+1)}$, although $Au_0 \rightarrow 0$.

Proof. Apply the finite geometric-sum identity to eq. (1). Since $u_0/(1 + Au_0) = 1/y$, the finite remainder has exactly the form (5). Dividing its magnitude by u_0^{n+2} gives (6). The final assertion follows from $A/y \rightarrow 0$. $\square$

In particular, $A = \sqrt{y}$ gives an actual error of order $y^{-(n+3)/2}$, whereas the reported scale is of order $y^{-(n+2)}$. This second family rules out dismissing the defect as only a borderline convergence example. The constants in the claimed source-coordinate theorem are being allowed to depend on the variable that is tending to its limit.

3.5 The valid fixed-data control

If A is a genuine fixed real constant, the same algebra yields a bounded normalized error:

$$\frac{|S_n - 1/y|}{u_0^{n+2}} = |A|^{n+1} \left(1 - \frac{A}{y}\right)$$

on a sufficiently large positive ray. For $y \geq 2|A|$ this is at most $\frac{3}{2}|A|^{n+1}$. A looser factor two is used in the independent tests. The $A = 0$ control is exact and follows the constructor's empty-perturbation path.

The correct fix must therefore distinguish fixed cancellations from moving cancellations. Rejecting every decomposition in which a constant cancels would be unnecessary and would remove mathematically valid inputs.

3.6 Native isolation: what is confirmed

The successful isolated Wolfram 15 evaluation used the upstream `corePerturbationTerm` body, changing only resource-failure plumbing to a local throw in a branch that was not reached. Its observed values included:

```
Total forward expression:      1/x
FreeQ[core + perturbation, y]:  True
FullSimplify[Element[perturbation, Reals]]: True
FullSimplify[u0, y>0]:        2/y
Marker coefficients, degrees 1..5:
{-2/y, 2/y, -2/y, 2/y, -2/y}
Finite sums, depths 0..5:
{2/y, 0, 2/y, 0, 2/y, 0}
```

This confirms the cancellation gate, the native coefficient-realness control, and the recurrence mechanics. It does *not* establish that the entire constructor or the complete patched package was executed. The integration probe included in this archive is the next required acceptance step, not evidence that it already ran.

4 Repair, regression design, and dual-runtime implications

4.1 Minimal semantic change

The essential repair is to replace the dependency check by

```
If[x === y || ! FreeQ[{core, perturbation}, y],
  fail["InvalidVariables",
    "Use distinct source and target symbols, with no target symbol in either core or
    perturbation."]];
```

The list preserves which expression supplied the data. Ordinary evaluation inside each operand may still remove a mathematically irrelevant occurrence; the important point is that addition must not erase a dependency *between* two operands that the algorithm keeps separate. `FreeQ` is a structural query on its evaluated expression; it cannot recover information already erased by prior evaluation.[11]

The companion change

```
validateInput[{core, perturbation}, limit];
```


keeps the exact-input and nonfinite-input checks on the same semantic unit. For example, inexact constants can cancel in the sum even though the core model still receives them separately. This companion change is an explicit strengthening of operand admission, not a separate counted bug and not a claim that every such input previously produced a wrong answer.

The patch retains the existing failure tag `InvalidVariables`. The total forward function may still be stored as `core + perturbation`; it is the right expression for the defining equation after both operands have passed admission. There is no need to change the valid fixed-data marker formula, change the result head, or reclassify this object as an exponent-sorted jet.

4.2 Why an output-only repair is inadequate

Three apparently reasonable postconditions would all miss the first witness: the total forward expression is target-free, the exact core inverse is free of the eliminated source symbol, and every displayed coefficient is real on the target ray. The obstruction belongs to the model's quantifiers, not to the syntactic cleanliness of the final expression.

Likewise, multiplying the first omitted term by a guessed numerical constant would not repair the declared remainder theorem. The all-depth ratio proof shows that a constant independent of the target does not exist for the stored scale. A general moving-parameter theory would need its own asymptotic grading, but the existing public contract calls for rejecting this input before invoking a fixed-data theorem. That much narrower change is the recommended repair.

4.3 Regression expectations

The supplied `tests/CoreOperandScope.wlt` is a desired-contract MUnit file. It is unrun. It requires both moving-coefficient families to return the structured variable-admission failure, keeps a fixed cancellation as a `GeneralizedSeries`, checks the exact zero-perturbation control, and checks that inexact operands do not disappear before the exact-input boundary.

The separate `ProbeCoreOperandScope.wl` is suitable for observational use in either session after loading the package. It prints the runtime and explicitly distinguishes a series result from a refusal. For a series it reads the actual `CoreInverse` fields `Expression`, `RemainderPower`, `RemainderScaleExpression`, and `TargetDomain`. It does not reuse an ordinary-forward-field assumption or count an unevaluated numerical guard as a passing comparison.

A useful native acceptance sequence is: run the probe on the original pin; apply the two-line semantic repair to a separate working copy; rebuild the standalone using the repository's maintained procedure; rerun the probe and MUnit file; then execute existing exact-core controls. The patch emitter does not perform these steps or claim their results.

4.4 Mathics: common source is not common observed behavior

The offending dependency check is shared source, so it should be corrected independently of which runtime first reproduces the bad public call. However, the particular `Abs[y]` witness relies on a realness fact that native Wolfram can discharge. The Mathics assumption adapter has a more conservative recursive realness prover; it may refuse the expression before reaching the same public result. Reading common source is not evidence that this public witness succeeds identically in Mathics.[4]

The precise claims are therefore: the shared guard loses operand information; the native isolated realness and recurrence controls pass; the full Wolfram manifestation follows the inspected source path but was not integration-run; and Mathics public reachability remains unmeasured. The

proposed list-based admission itself uses elementary operations already present in the sibling implementation and does not introduce a new special-function dependency.

The Lerch prototype in section 5 is intentionally separate. It uses exact rational operations, finite loops, lists, and a small association wrapper. That is a design for portability, not proof of runtime compatibility. A fresh Mathics run of the supplied file is still required.

5 E01: a rational Lerch certificate hierarchy

5.1 The bounded target

For fixed $0 < q < 1$ and $a > 0$, define

$$F_q(a) = \Phi(q, 1, a) = \sum_{k=0}^{\infty} \frac{q^k}{a+k}. \quad (7)$$

This is the $s = 1$ slice of the defining Lerch series.[12] The proposed primitive returns an exact interval for the *function value*, not a replacement remainder for an arbitrary generalized-series object. Its first implementation accepts rational q, a and a positive integer order. It treats $q = 0$ separately, where $F_0(a) = 1/a$ exactly.

The restriction is useful. A positive discrete measure is available, all moments are finite, and both denominator families have explicit positive evaluations on the negative real axis. No nonprincipal branches or numerical sign decisions enter the certificate. Extending to $q < 0$, complex parameters, or arbitrary s is outside this prototype.

5.2 Two positive measures, not a moment matrix

Let

$$\mu = \sum_{k \geq 0} q^k \delta_k, \quad \mu_1 = \sum_{k \geq 0} k q^k \delta_k.$$

Their masses are

$$m_0 = \frac{1}{1-q}, \quad m_1 = \frac{q}{(1-q)^2}. \quad (8)$$

Then $F_q(a) = \int (a+t)^{-1} d\mu(t)$. The weighted transform satisfies

$$H_q(a) = \int \frac{1}{a+t} d\mu_1(t) = m_0 - a F_q(a). \quad (9)$$

The first measure has support in $[0, \infty)$; the second has support in $[1, \infty)$ after its zero weight at $k = 0$ is discarded. This one-unit shift will improve its elementary error cap.

The first measure is the Meixner weight with parameter $\beta = 1$. Writing $k = t + 1$ in the second gives $q(t+1)q^t$, namely q times the Meixner weight with $\beta = 2$, translated by one. The defining Meixner orthogonality and finite hypergeometric representation are recorded in DLMF §§18.19–18.20.[13, 14]

5.3 A general resolvent identity

Let P_n be the monic degree- n polynomial orthogonal for a positive measure ν supported in $[b, \infty)$, where $b \geq 0$. Define

$$G_n^\nu(a) = \frac{1}{P_n(-a)} \int \frac{P_n(-a) - P_n(t)}{a+t} d\nu(t). \quad (10)$$

The quotient in the integral is a polynomial of degree $n - 1$, so this is a rational function computable from finite polynomial information. The formula also identifies the usual Gaussian resolvent approximant without requiring its quadrature nodes.

Theorem 5.1 (Positive exact error identity). *Assume $a > 0$ and $P_n(-a) \neq 0$. Then*

$$\int \frac{1}{a+t} d\nu(t) - G_n^\nu(a) = \frac{1}{P_n(-a)^2} \int \frac{P_n(t)^2}{a+t} d\nu(t). \quad (11)$$

In particular the error is positive for a nondegenerate measure, and with $h_n^\nu = \int P_n^2 d\nu$ it is at most

$$\frac{h_n^\nu}{(a+b)P_n(-a)^2}. \quad (12)$$

Proof. The polynomial $(P_n(t) - P_n(-a))/(t+a)$ has degree $n - 1$. Orthogonality gives

$$0 = \int P_n(t) \frac{P_n(t) - P_n(-a)}{a+t} d\nu(t).$$

Rearranging yields $\int P_n(t)/(a+t) d\nu(t) = P_n(-a)^{-1} \int P_n(t)^2/(a+t) d\nu(t)$. Subtract eq. (10) from the original transform and substitute this identity. Positivity follows from the integrand. Finally $a+t \geq a+b$ gives eq. (12). $\square$

5.4 The lower and upper endpoints

Use $G_n = G_n^\mu$ and $J_n = G_n^{\mu_1}$, and set

$$L_n(a) = G_n(a), \quad U_n(a) = \frac{m_0 - J_n(a)}{a}. \quad (13)$$

By eqs. (9) and (11),

$$L_n(a) < F_q(a) < U_n(a) \quad (0 < q < 1, a > 0). \quad (14)$$

The upper endpoint is the Gauss/Radau-type bound obtained by applying the Gaussian lower-bound construction to the size-biased measure. Both endpoints are rational when q, a are rational.

This avoids a common integration mistake: an accurate lower approximation to H_q becomes an *upper* approximation to F_q because of the minus sign in eq. (9). The implementation keeps the two measures separate and its independent oracle checks both orientations.

5.5 Closed recurrence coefficients

For a Meixner parameter $\beta > 0$ translated by $\delta \geq 0$, the monic three-term recurrence has coefficients

$$b_j = \delta + \frac{(1+q)j + \beta q}{1-q}, \quad (15)$$

$$\lambda_j = \frac{qj(j + \beta - 1)}{(1-q)^2}, \quad j \geq 1. \quad (16)$$

They follow by substituting the finite Meixner representation into the monic recurrence, or by matching its polynomial coefficients. The independent tests also reconstruct the finite polynomials and their exact moments, rather than using this recurrence as their sole oracle.

For total mass m , initialize

$$D_0 = 1, \quad D_1 = a + b_0, \quad N_0 = 0, \quad N_1 = m.$$

For $j = 1, \dots, n-1$, compute

$$D_{j+1} = (a + b_j)D_j - \lambda_j D_{j-1}, \quad (17)$$

$$N_{j+1} = (a + b_j)N_j - \lambda_j N_{j-1}. \quad (18)$$

The rational approximant is N_n/D_n . The denominator is $D_n = (-1)^n P_n(-a)$. For L_n , use $(\beta, \delta, m) = (1, 0, m_0)$; for J_n , use $(2, 1, m_1)$.

Only two consecutive numerator and denominator values are needed. For a single requested order n , two recurrences cost $O(n)$ exact arithmetic operations and $O(1)$ live scalar slots. This is not an $O(n)$ bit-complexity claim: the numerators and denominators grow, and rational reductions have nonconstant cost. The proposal nevertheless removes moment-matrix construction, linear-system solution, root isolation and special-function evaluation from this certificate's production path.

5.6 Denominator positivity without approximate roots

A subtraction recurrence deserves a denominator proof even when sample outputs look positive. The finite Meixner formula gives

$$D_n(a; \beta, \delta) = (\beta)_n \left(\frac{q}{1-q} \right)^n \sum_{j=0}^n \binom{n}{j} \frac{(a+\delta)_j}{(\beta)_j} \left(\frac{1-q}{q} \right)^j. \quad (19)$$

Every summand is positive in the stated parameter domain. Thus $D_n > 0$ exactly, without computing a root or rounding a sign. The production code additionally checks its positive-denominator invariant to catch an implementation error; the mathematical guarantee is eq. (19), not that runtime check.

For the two required parameter choices, all factors are rational at rational a, q . Although eq. (19) is also an evaluation algorithm, the recurrence is used in production; the finite sum is an independent exact check and a convenient proof certificate.

5.7 Nested bounds

Let $W_n = N_n D_{n-1} - N_{n-1} D_n$. Equation (18) gives

$$W_{n+1} = \lambda_n W_n, \quad W_1 = m.$$

Consequently

$$\frac{N_{n+1}}{D_{n+1}} - \frac{N_n}{D_n} = \frac{m \lambda_1 \cdots \lambda_n}{D_n D_{n+1}} > 0. \quad (20)$$

The lower endpoints increase. The Gaussian approximants to H_q also increase, so eq. (9) makes the upper endpoints decrease:

$$L_n < L_{n+1} < F_q < U_{n+1} < U_n.$$

Thus increasing the order retains the previous enclosure while shrinking its width. This is a property proved for the exact construction, not an empirical rule inferred from the numerical examples.

5.8 Explicit error caps and asymptotic order

The monic squared norms are

$$h_n = \frac{(n!)^2 q^n}{(1-q)^{2n+1}}, \quad (21)$$

$$h_n^{(1)} = \frac{n!(n+1)!q^{n+1}}{(1-q)^{2n+2}}. \quad (22)$$

These follow from the Meixner normalization, or from $h_n = m \prod_{j=1}^n \lambda_j$ with the two parameter choices.[13] Writing $D_n^{(1)}$ for the denominator of J_n gives the separately valid caps

$$0 < F_q - L_n \leq \frac{h_n}{aD_n^2}, \quad (23)$$

$$0 < U_n - F_q \leq \frac{h_n^{(1)}}{a(a+1)(D_n^{(1)})^2}. \quad (24)$$

The factor $a+1$ uses the actual support of μ_1 ; replacing it by a would remain sound but lose information.

At fixed q, n , the exact error identities also give

$$F_q - L_n = h_n a^{-2n-1} + O(a^{-2n-2}), \quad (25)$$

$$U_n - F_q = h_n^{(1)} a^{-2n-2} + O(a^{-2n-3}). \quad (26)$$

To justify these statements, D_n is monic in a , and the difference between $\int P_n(t)^2/(a+t) d\nu$ and h_n/a is bounded in magnitude by $a^{-2} \int t P_n(t)^2 d\nu$. The required moment is finite. No uniformity as $q \rightarrow 1$ is claimed; the displayed norms explicitly reveal the deterioration near that endpoint.

5.9 Concrete exact example

For $q = 1/2$, $a = 10$, the first four intervals are:

n	L_n	U_n	$U_n - L_n$
1	2/11	12/65	2/715
2	7/38	47/255	1/9690
3	115/624	857/4650	1/161200
4	1087/5898	8863/48090	4/7878745

The corresponding width displays are approximately $2.80 \cdot 10^{-3}$, $1.03 \cdot 10^{-4}$, $6.20 \cdot 10^{-6}$, and $5.08 \cdot 10^{-7}$. The fractions, not these decimal displays, are the certificates. At order two the additional caps are

$$F_{1/2}(10) - \frac{7}{38} \leq \frac{1}{7220}, \quad \frac{47}{255} - F_{1/2}(10) \leq \frac{1}{47685}.$$

The isolated Wolfram 15 recurrence evaluation returned precisely the four rational endpoint pairs and their error caps. Four native comparisons with `LerchPhi[1/2,1,10]` returned `True`. The independent exact oracle provides a separate check not based on that built-in function.

6 Implementation contracts and tests for E01

6.1 A small API with explicit semantics

The Python entry point is

```
lerch_bounds(q: int | Fraction,
            a: int | Fraction,
            order: int) -> LerchBounds
```

It returns exact endpoints, their width, two separate error caps, the requested order, the input values, and a method identifier. Approximate inputs, strings, booleans used as numbers, invalid domains, and noninteger orders are rejected. The prototype caps the order at 64 as an explicit resource policy; the theorem is not limited to 64. Its degenerate $q = 0$ branch returns an exact point interval.

The WL counterpart is `ReviewLerch` `LerchRationalBounds[q,a,n]` in a separate context. It returns a small association or a structured failure. It does not install definitions on `System` symbols, alter the repository's private helpers, or modify `LerchPhi`. The supplied wrapper file has not been run as a complete file in either runtime; the native observation executes the same arithmetic recurrence body with an isolation wrapper.

6.2 Why value bounds must not become series metadata accidentally

An enclosure of $F_q(a)$ is not automatically an error bound for an arbitrary stored truncated expression $T(a)$. At a given exact parameter value, the valid transport is

$$F_q(a) - T(a) \in [L_n(a) - T(a), U_n(a) - T(a)].$$

An absolute error bound is then the maximum of the absolute values of these two endpoints, provided $T(a)$ itself has been evaluated exactly or enclosed independently. A decimal approximation to $T(a)$ is not sufficient for that step. The new primitive must not silently declare a derivative contract, a uniform parameter-domain theorem, or an inverse-branch certificate.

A first integration should therefore remain an opt-in value-certificate helper. It can attach its exact input point, theorem family, order, endpoints and arithmetic domain to a result. Promotion into more general remainder metadata should be a separate, explicitly tested transport operation. This is a concrete contract for this new family, not a replacement proposal for the repository's already-recorded result-schema work.

6.3 Independent validation paths

The production recurrence is checked in three different ways. First, the defining sum gives the entirely rational enclosure

$$S_K \leq F_q(a) \leq S_K + \frac{q^K}{(a+K)(1-q)}, \quad S_K = \sum_{k=0}^{K-1} \frac{q^k}{a+k}. \quad (27)$$

The test increases K until this oracle interval lies inside the proposed interval and is fine enough to verify both explicit error caps.

Second, test-only code constructs Meixner polynomials from their terminating finite formula. It forms the polynomial quotient in eq. (10) and integrates it using exact factorial moments. Those moments are obtained from

$$\sum_{k \geq 0} \frac{(\beta)_k}{k!} q^k k^j = \frac{(\beta)_j q^j}{(1-q)^{\beta+j}}.$$

A Stirling-number conversion supplies ordinary moments. This is a different construction from the production three-term recurrence.

Third, the tests verify orthogonality and squared norms exactly, compare the positive finite denominator sum with the recurrence, and check interval nesting. These are algebraic identities, not floating-point residual thresholds.

Selected subcase population	Count
Defining-sum containment and error-cap cases	144
Exact orthogonality identities	224
Moment/quotient endpoint comparisons	144
Adjacent-order nesting comparisons	264
Moving-parameter inverse cases	66
Largest defining-sum cutoff needed	512

These populations belong to the 24 unittest methods; they are not additive acceptance counts. The parameter grid for containment is $q \in \{1/5, 1/2, 3/4, 9/10\}$, $a \in \{1/2, 1, 2, 10, 100, 1000\}$ and $n = 1, \dots, 6$.

6.4 A test-harness correction, preserved in the evidence

The first Python pass had two subcase failures. The oracle-refinement loop initially stopped as soon as its interval lay inside $[L_n, U_n]$, then tried to verify the tighter individual error caps with the same insufficiently refined oracle. The endpoint-enclosure tests had succeeded; the oracle did not yet have enough resolution to establish the separate cap inequalities.

The correction was to require all four inequalities in the loop's stopping condition. Neither the recurrence nor the mathematical caps were changed. The final pass has 24 successful methods and no failures. Both first-pass and final records are included so that the improvement in the test harness is not misrepresented as an unexplained change in mathematical output.

6.5 Performance claims that are and are not supported

For each measure the implementation performs $n - 1$ recurrence steps after initialization, and it updates a norm product at the same time. It keeps only two consecutive values of each recurrence. Those structural work bounds follow from the source and can be inspected directly.

No Wolfram-versus-Mathics timing comparison was run. There is no claim that this prototype outperforms the existing package on a measured workload, that order 64 is an optimal cap, or that exact arithmetic prevents all expression growth. Near $q = 1$ or with large rational input heights, numerator growth can dominate. A production implementation should expose the cost of this particular finite certificate separately from the approximation order of an unrelated series.

7 Other inspected candidates not counted as findings

A careful incremental audit needs to discard attractive but unsupported claims. One examined path was the terminal branch of the private exponent comparator, which can infer a positive ordering from a simplified negation of a negative comparison. In isolation, $\neg(d < 0)$ only implies $d \geq 0$, not $d > 0$. However, the actual comparator first canonicalizes its exact inputs and checks for zero. A public counterexample must survive that earlier stage and satisfy the caller's admitted input domain; a purely symbolic private-helper example would not establish this.[3]

Two proposed noncanonical-zero witnesses were rejected during this audit. A Bessel Wronskian combination and a rational-argument dilogarithm five-term identity both reduced to zero under the available native `FullSimplify` control. The latter evaluation returned a canonical absolute difference of zero. That canonical zero is decisive: it prevents the alleged path through the comparator’s final branch. No second public ordering defect or repair is claimed from those examples.

This matters for the evidence standard. A suspicious implication in a helper, a failed direct numerical comparison, and a mathematical identity are not enough when the real control flow has an earlier successful normalization. The article therefore retains N01 as its one new correctness finding rather than inflating the count with an unestablished public manifestation.

8 Integration priorities and reproducibility

8.1 Recommended sequence

The first change should be the operand-preserving N01 admission rule, with the moving-small family as well as the alternating family in acceptance. The valid fixed-cancellation and exact-zero controls are essential: they prevent an unnecessarily broad rejection policy.

The second stage is a focused execution of those requests in each runtime. A Mathics refusal caused by a separate realness limitation should be recorded as such, not described as proof that the underlying shared admission is sound. The generated standalone and modular loading routes should agree after rebuilding. This report’s source patch has not established that agreement.

The rational Lerch primitive should be integrated separately. First run the supplied WL wrapper on the same exact fixtures as the Python implementation. Then attach it only to the $s = 1$, $0 \leq q < 1$, $a > 0$ value-certificate path. Broader symbolic parameters or automatic dispatch should wait for a separately stated admission and transport contract. This sequencing prevents a small correctness repair from being entangled with an unrelated algorithmic extension.

8.2 Archive layout

```
article/article.tex
article/article.pdf
code/emit_core_patch.py
code/lerch_rational.py
code/LerchRationalBounds.wl
tests/test_review.py
tests/run_tests.py
tests/CoreOperandScope.wlt
tests/ProbeCoreOperandScope.wl
evidence/...
README.md
build.sh
LICENSE
```

The archive contains no checksum files and no complete copy of the upstream repository. The patch emitter reads a local checkout supplied by the reader and emits a unified diff without modifying source. It requires each exact source anchor to occur once; missing, duplicated, or already-patched anchors are rejected. Its tests use synthetic fixtures and do not prove that a full checkout was patched successfully.

8.3 Commands

From the archive root, the independent tests require only a standard Python installation supporting the supplied syntax:

```
python tests/run_tests.py
python code/lerch_rational.py 1/2 10 4
python code/emit_core_patch.py /path/to/Asymptotic --output core-operands.patch
```

The test runner regenerates the final evidence files; preserve the supplied records elsewhere before overwriting them for a new environment. The initial historical pass is retained separately.

For a symbolic session, load the files explicitly:

```
Get["/path/to/archive/code/LerchRationalBounds.wl"];
ReviewLerch[LerchRationalBounds[1/2,10,4]

Get["/path/to/Asymptotic/AsymptoticAnalysis.wl"];
Get["/path/to/archive/tests/ProbeCoreOperandScope.wl"];
```

Use the repository's own rebuilding instructions after applying a patch to its modular sources. A generated standalone must not be assumed current merely because the modular file was edited. This archive does not automate or assert that repository-specific rebuild.

The article builds with three `pdflatex` passes; `build.sh` runs them in a temporary directory and copies only the final PDF into `article/`. No bibliography program, network request, or proprietary font is required.

9 Conclusion

The new correctness result is a failure of *operand-level admission*, not of the marker coefficient formula. Checking the simplified sum establishes a property of the total forward function but not of the separately retained model data. The exact counterexample shows why that distinction matters: the defining function is elementary, its inverse is unambiguous, and yet the stored asymptotic scale is false at every depth. A genuinely small moving perturbation produces the same contract failure, so convergence screening alone cannot repair it.

The rational Lerch proposal illustrates a complementary development direction: identify a narrow mathematical family, derive the certificate including its denominator and remainder obligations, and implement the arithmetic without requiring the symbolic kernel to rediscover those facts. Here two explicit Meixner recurrences provide a nested exact interval and separately bounded endpoint errors. The independent tests and isolated native controls support that prototype at their stated scopes. Full package integration and Mathics execution remain concrete acceptance work, not results claimed by this review.

A Source and evidence map

Claim	Primary evidence
N01 sum-only admission	<code>corePerturbationConstruct</code> in <code>CorePerturbation.wl</code> ; pinned source in reference [1].
Separate model operands	<code>corePerturbationModel</code> , including separate <code>rows</code> , <code>remainderRows</code> , and <code>Offset</code> .

Claim	Primary evidence
Exact inverse and marker terms	<code>corePerturbationAutomaticInverse</code> , <code>corePerturbationTerm</code> , and constructor assembly.
Remainder theorem quantified over fixed data	Constructor <code>truncationPair</code> , <code>scale</code> , and <code>MajorantContract</code> .
Native recurrence observation	<code>evidence/native-observations.json</code> , record N01-isolated-recurrence; transcribed selected tool output.
Native rational prototype observation	Same file, record E01-isolated-recurrence; not a complete wrapper-file or package run.
Independent models and fixtures	<code>tests/test_review.py</code> , <code>evidence/python-results.json</code> , and <code>python-tests.txt</code> .
First test-oracle pass	<code>python-first-pass.json</code> and <code>python-first-pass.txt</code> ; two subcase failures before oracle refinement.
Exact rational sample intervals	<code>evidence/lerch-exact-samples.json</code> . Decimal values in the CSV are display-only.
Novelty and omissions	<code>evidence/novelty-ledger.json</code> and <code>evidence/scope.json</code> .

References

- [1] V. Reshetnikov, *AsymptoticAnalysis*, `src/Kernel/CorePerturbation.wl`, reviewed commit `efalaeeec4845a9c35e140963a0333d0c9ec33b05`. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/CorePerturbation.wl>.
- [2] Same repository and commit, `src/Kernel/ExponentialCorePerturbation.wl`. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/ExponentialCorePerturbation.wl>.
- [3] Same repository and commit, `src/Kernel/AsymptoticAnalysis.wl`, exact-real canonicalization and comparison. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/AsymptoticAnalysis.wl>.
- [4] Same repository and commit, `src/Kernel/MathicsAssumptions.wl`, finite-real and sign proof rules. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/MathicsAssumptions.wl>.
- [5] Same repository and commit, review index and six linked wave indexes. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/README.md>.
- [6] Same repository and commit, `docs/development/CODE_REVIEW_STATUS.md`. https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/docs/development/CODE_REVIEW_STATUS.md.
- [7] Same repository and commit, wave-5 review index. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-5/README.md>.
- [8] Same repository and commit, wave-6 review index. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-6/README.md>.
- [9] Retained review 49, *Incremental Contract Audit of AsymptoticAnalysis*, section on signed Lerch remainder bounds, at the reviewed commit. <https://github.com/VladimirReshetnikov/>

[Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-6/code-review-49/article.tex](https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-6/code-review-49/article.tex).

- [10] Retained review 50, *Fixed charts, exact cores, rational moments*, section on Eulerian Lerch moments, at the reviewed commit. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-6/code-review-50/article/article.tex>.
- [11] Wolfram Research, *FreeQ*, Wolfram Language documentation, accessed 10 September 2026. <https://reference.wolfram.com/language/ref/FreeQ.html>.
- [12] NIST Digital Library of Mathematical Functions, §25.14, *Lerch's Transcendent*, accessed 10 September 2026. <https://dlmf.nist.gov/25.14>.
- [13] NIST DLMF, §18.19, *Hahn Class: Definitions*, Meixner weights, norms and leading coefficients, accessed 10 September 2026. <https://dlmf.nist.gov/18.19>.
- [14] NIST DLMF, §18.20, *Hahn Class: Explicit Representations*, Meixner finite hypergeometric representation, accessed 10 September 2026. <https://dlmf.nist.gov/18.20>.